Iso


ひなゲー②
4月19日発売です
SECOND EDITION
THE
C
ANSI
PROGRAMMING
LANGUAGE
店舗特典
・メロンブックス
・ゲーマーズ
・とらのあな
・アニメイト
・COMICZIN

CHAPTER 1: INTRODUCTION AND A TUTORIAL

Greetings, fellow C learner, the fact that you're reading this book means that you either wanna learn C from another C learner in the modern days (writing this book on the 1st October 2026), or you found the aesthetic appealing, and lowkey that's the exact reason why i made it, i was looking for a book with cute anime girls teachings and actual comprehensible up-to-date book that covers the essentials and is fun to learn…, it's also a remake to the infamous "Second edition: The C programming language" by *Brian Kernighan, Dennis M. Ritchie* .
Anyway, here i would be sharing my knowledge regarding C, as i slowly progress too along the way of writing this book, the book will cover a series of concepts, starting from the raw basics, going through variables and constants, arithmetic, control flow, functions, and the rudiments of input and output, and hopefully pointers and more advanced stuff eventually.
We might touch more concepts as we progress and learn.
The approach in use:
In this book we'll be doing real time exercises to make ourselves acquainted with the C syntax and to also tickle the compiler and debug errors if we got any, there would be also some theoretical parts, i know shocker, where we understand certain topics and acronyms like "malloc" (memory allocation) and a "buffer overflow", pretty interesting and important stuff to be honest…
And finally, i'm not the all knowing C expert at the moment, just an enthusiastic developer driven by meme culture and a heavy ego to learn it all, so if you seen something weird or you feel wrong feel free to let me know, i'll be tagging my socials on the book's ending…

1.1.1 Getting started: what is C

C is the godfather of all programming languages, and the Godmother of all operating systems, where C was created in 1972 by Dennis Ritchie to rewrite UNIX, it made Linux Kernel, parts of Windows kernel AND MacOS kernel, pretty cool stuff, it also made Cpython, or basically python language at this point, and mysql too , aside from DOOM and the Quake Engine for it, and finally it's used in cars too, YEP the computer insider your car, literally C is the shit, you can basically make anything with it.

1.1.2 Getting started: setting up your environment

Now here, i'll be guiding you based on what i know as a NixOS user… To set up your development environment, i advice you to install "clang" package short for "c language" and "gcc" the compiler for C, or just one of them is fine, but for some reason i got the itch to keep them both.
Choose an IDE you're comfortable in, VScode (install the extensions with it gang), ZED editor (made by the Rust folks, they're our lovely neighbours), or you can go with TUI like me, i got neovim…
It doesn't matter what you use as long as you got clang or gcc AND a terminal

So after you got all that, we'll be starting by creating our first C program, create a new file with the extension c, or just copy the commands (for linux)

Shell

```
mkdir new_C_program
cd new_C_program
touch main.c
```

And yeah start coding…
Our first hello world program:

C/C++

```
#include <stdio.h>                              // stands for "standard
input/output"
int main() {                                    // the entry to the
program
        printf("hewo meow meow!\n");                    // your kawaii
printing function
        return 0;                               // to say "hurraaay
success"
}
```

NOW compiling the program to see how to run your code:

1. Go to your terminal and type: gcc main.c
2. A new file will appear named a.out
3. Run it using: ./a.out
4. You're supposed to see: hewo meow meow , or anything you wrote

To compile the code you wrote to any file name you wish do this

1. Terminal again: gcc main.c -o file_name
2. You see the name of the program

Now we're done with the cultural norms when learning a programing language

=> your homework: write a program that prints a lot of statements, maybe a kawaii ASCII drawing you do to theme your terminal or something, one thing tho `print(“\n”);` let’s you create another line or perhaps let’s say equivalent to hitting the return button

1.2 The Basics: Over view

Moving to variables, now you see this shit is reaaaally important you see, unlike other high level languages, C does not hold your hand when you for example type `number = “six seven” ` a high level language *uhm python uhm* will save the number as a string
And what is a string you ask?
A string is at its core a series of characters, and what’s a character you said?
A character is a type of data with each character the ability to hold 8 bytes of data… now we have to stop and think about what’s a byte and why exactly 8?

We all know that our dear lovely machines cant understand human spoken words, but only 1s and 0s, or more precisely, to whether it has electricity(signal) or not
So what the human race did was to give those signals a meaning and shuffle them into symbols, this table will give u ideas

8-Bit Binary Alphabet Reference (FIXED)

Char	Dec	8-Bit Binary
A	65	01000001
B	66	01000010
C	67	01000011
D	68	01000100
E	69	01000101
F	70	01000110
G	71	01000111
H	72	01001000
I	73	01001001
J	74	01001010
K	75	01001011
L	76	01001100
M	77	01001101
N	78	01001110
O	79	01001111
P	80	01010000
Q	81	01010001
R	82	01010010
S	83	01010011
T	84	01010100
U	85	01010101
V	86	01010110
W	87	01010111
X	88	01011000
Y	89	01011001
Z	90	01011010

Printable Latin alphabet, uppercase, standard ASCII/ISO-8859-1 encoding. Corrected binary values.

It might look overwhelming at first but it's really easy, each character has its unique 8 bit binary, and a decimal number, to make it into perspective
00000000 (binary) = 1 (decimal); 00000001 = 2 ; 00000010 = 3 ; 00000011 = 4
And so on until you reach your 01000001 = 65
So that's how the computer views your text, pretty cool, you can larp your way across the world using this, like this:
01001001 01001100 01001111 01010110 01000101 01011001 01001111 01010101
Your homework is to decode this and translate it.

Now that we know what's a byte and what it can do to make our data types, we shall move on!

1.2.1 Variables & Data Types

Basic Types

In C, data types specify how memory is allocated and how stored bits are interpreted by the compiler. Here is a breakdown of the standard primitive data types:

NOTE: Float vs. Double Research & Comparison

- **float**: Single-precision floating-point number. Takes 4 bytes of memory and offers ~6-7 decimal digits of precision. Use when memory is tight and high precision is not required. Example: `float weight = 60.27f;` (note the 'f' suffix for float literals).
- **double**: Double-precision floating-point number. Takes 8 bytes of memory and provides ~15-17 decimal digits of precision. This is the default choice in C for floating-point values. Example: `double gpa = 4.5;`

Here are the primary types you'll use:

- **int (Integer):** Stores whole numbers without decimals (typically 4 bytes, range -2,147,483,648 to 2,147,483,647).

```
C/C++
int age = 20; // 4-byte signed integer
```

- **float (Single-precision float):**

```
C/C++
float weight = 60.27f; // note the 'f' suffix for float literals
```

- **double (Double-precision float):**

```
C/C++
double gpa = 4.5; // standard double precision
```

- **bool (Boolean):** Represents boolean truth values (true or false). Requires including <*stdbool.h*> in standard C99 and later.

```
C/C++
bool is_happy = true; // 1 byte (requires <stdbool.h>)
```

- **char (Character):** Stores a single character or ASCII character code (1 byte). Always enclosed in single quotes!

```
C/C++
char grade = 'A'; // enclosed in single quotes
```

- **char[n] (Character Array / String):** C does not have a native string type; strings are represented as null-terminated character arrays.

```C/C++
char name[6] = "nyxie"; // 5 characters + 1 null terminator ('\0')
```

1.2.2 Variable Output

Now now, as you saw before, you'll have to declare the type of the variable you want and be precise with it, and because C is such a "Bitch, i control everything" ahh language, even in output, you'll have to specify exactly the type of data you want to display, and that's IMPORTANT! Because it doesn't only depend on output, but A LOT LOOOT more, anyway try to memorize this table:

Data Type	Output Specifier
Integer (`int`)	`%d`
`double`	`%lf` or `%f`
`float`	`%f`
Boolean (`bool`)	`%d` (because again, it's 1 or 0)
`char`	`%c`
`char[n]`	`%s`

So if you want to combine all the skills you've learned so far in this section, you'd prolly have something like this:

```C/C++
#include <stdio.h>
#include <stdbool.h> // standard library header required for booleans in C

int main() {
    char name[6] = "nyxie";
    int age = 20;
    float height = 175.6355442f; // we'll do something with this precision
    bool is_kawaii = true;
    char blood_type = 'O';

    printf("hello my name is %s i'm %d years old, my height is: %.2f, my blood type is under the %c family, am i kawaii?: %d\n", name, age, height, blood_type, is_kawaii);
    return 0;
}
```

Now now, the output shall be something like:
“hello my name is nyxie i’m 20 years old, my height is: 175.64, my blood type is under the O family, am i kawaii?: 1”

And I know, I know, you must’ve already caught a couple of interesting things, like:

- Why is `is_kawaii` not “true” but instead `1`?
- Why is height only `175.64` and not the full thing?

And your questions are valid, pookie, I like it when you ask questions…
Anyway, we all know that booleans are true or false, or rather… 1 or 0, so keep that in mind, you’d need it in future projects… If something is true it returns a 1 signal, if it’s false, it returns a 0.

As for the float, you can control the amount of digits you want to display in your output…

- Let’s say for example we have a float number like `15.223f`.
- If I only wanted to display 2 decimal places, I’m gonna use `%.2f`.
- If I have `3.14159265358979`, and I want to display the first 5 digits after the dot, it’ll be `%.5f`.
- And if I have for example `20061.10420` and want a minimum width of 2 with 3 decimal places, it’ll be `%2.3f` and so on. It’s all up to you, pookster!

Now, your exercise for today… play around with your program! Just write, write, and have fun, make silly things, make dumb programs, and get familiar with the syntax…
If you want to be more oriented, then heheee, make a program that introduces you, and your friends if you have any 👀

1.2.3 Data Input

Now reaching somewhere exciting: data input! How to take data from the user, or rather how to fill up those variables without the need to hardcode them in your program.
There are a lot of ways to take input, and we’re walking through all of them because it’s important…

Declaring Placeholders

However first, we’ll have to declare our variables:

```C/C++
int main() {
    int number;
    char letter;
    int something;
    return 0;
}
```

That’s how you give placeholders for those variables!

1.2.3.1 Scanf

```
C/C++
       scanf("%d", &num);
       scanf(" %c", &let);
       scanf("%d", &num2);
```

The Buffer Gotcha (Whitespace & Newlines)

Now this code snippet is rather interesting, because why is there a space before `%c`? Hummm… well apparently, `%c` is rather a greeeeedy one! If you hit enter before it (as in entering the input before that), it grabs the ENTER character (`\n`). That's crazy, so you put a leading space there to tell it: "ayo listen, skip any whitespace or newline, lock in and take what's after!"

The Address-Of Operator (&)

And also, what the helly is the & sign when assigning the input?
The & is called the address-of operator, and now you're sniffing some memory again! Basically, it tells the compiler to point to the exact memory address (which looks something like `0x7ffee3b8`). `scanf()` specifically needs that because it's like telling `scanf`: "hey, go to this specific memory address and put the input value right there."

Exceptions: Character Arrays & Strings

And just like that, it works! BUT… there is an exception, which is character arrays (strings):

```
C/C++
       scanf("%s", sumthin);
```

Here you do not have to write &, BECAUSE array variables already decay into pointers to the very memory address you're writing to! Pretty cool, huh? We'll see more stuff about it later.

1.2.3.2 Getchar()

This one's pretty lovely and straight forward, it only takes a single character... I know, shocking!
Anyway, here is its syntax:

```
C/C++
       #include <stdio.h>

       int main() {
           char halo;
           printf("Enter your character: ");
```

```
    halo = getchar();
    printf("%c\n", halo);
    return 0;
}
```

Just like that, like assigning the character to a function hummm~

1.2.3.3 fgets()

This one's a bit of a bomboclat function—it's safe and professional, but I find it a bit confusing at times. It reads an entire line of input safely!

C/C++

```
#include <stdio.h>

int main() {
    char batman[50];
    printf("Enter your full name: ");

    // Syntax: fgets(buffer, max_size, input_stream)
    fgets(batman, sizeof(batman), stdin);

    printf("Hello, %s", batman);
    return 0;
}
```

"buffer" is just the block's name that you'll store your shit in—it sounds big but it's not!
`sizeof()` is a function that retrieves the size of that buffer.
`stdin` is just setting the mode: standard input.

1.2.3.4 sscanf()

Pretty interesting one! Advanced a bit logically, but quite enticing if executed right. You can assign variables on the go, pretty cool shii~

C/C++

```
#include <stdio.h>

int main() {
    char input[] = "Nyx 20"; // prefilled character array
    char name[10]; // hehee preparing for a name that could contain 10
letters
    int age; // preparing the age integer

    // Parses data out of the input string
```

```
    sscanf(input, "%s %d", name, &age); // holy beans, taking in the
already assigned character array, and giving values to the rest of the
variables… pretty interesting

    printf("%s is %d years old.\n", name, age);
    return 0;
}
```

OKAY! For now those are the main simple ways to take input, there are other cooler ways that we'll see later for much more advanced stuff, but for now, this is all you need to know at the moment.

1.2.4 Operators

Operators in C are as easy as beans, but there are a few sneaky gotchas you need to keep your eyes on! Let's break them down topic by topic~

Arithmetic Operators

- **+** : Addition
- **-** : Subtraction
- ***** : Multiplication (or pointers 👀 *we'll check it later pookie*)
- **/** : Division
- **%** : Modulo (gives you the remainder of an integer division!)
- ****** : Wait a second... **C DOES NOT HAVE A POWER OPERATOR!** If you're coming from Python, don't try typing `2 ** 3` because the compiler will yell at you! In C, `**` is actually a pointer-to-pointer (double pointer 👀 *pookie you'd love it when we get there*). If you want powers, you gotta include `<math.h>` and use `pow(base, exp)`!

Increment & Decrement Operators

Need to quickly bump a number up or down by 1? C gives us super cute shorthand syntax for that:

- **++** : Increment (adds 1 to the variable)
- **--** : Decrement (subtracts 1 from the variable)

```C/C++
int energy = 10;
energy++; // energy is now 11 ✨
energy--; // energy is back to 10!
```

Assignment Operators

Store or update values in a flash with these assignment shortcuts:

- **=** : Basic assignment (assigns right value to left variable)
- **+=** : Add and assign (e.g. `x += 5` is shorthand for `x = x + 5`)

- `-=` : Subtract and assign
- `*=` : Multiply and assign
- `/=` : Divide and assign
- `%=` : Modulo and assign

Relational & Equality Operators

Used for comparing values! They evaluate to true (1) or false (0):

- `<` : Lesser than
- `>` : Greater than
- `<=` : Lesser or equal to
- `>=` : Greater or equal to
- `==` : Is equal to
- `!=` : Is not equal to

⚠️ **THE CLASSIC '=' VS '==' GOTCHA:**
Single '=' is for ASSIGNMENT (setting a value). Double '==' is for EQUALITY CHECK (comparing two values). If you write `if (x = 5)` instead of `if (x == 5)`, you are overwriting x with 5 and making the condition always true! Be super careful, pookster!

Logical Operators

Combine multiple conditions together:

- **`&&`** : Logical AND (both conditions must be true)
- `||` : Logical OR (at least one condition must be true)
- `!` : Logical NOT (flips true to false and vice-versa)

1.2.5 Conditions

FINALLY! The people's favorite, our love and joy, and where the fun and logic testing start…
Just like what the name says, conditions allow executing instructions if something meets the condition's expectations. There are three primary variations of a condition:

1. `if (condition) { instruction }`
2. `if (condition) { instruction } else if (condition) { instruction }`
3. `if (condition) { instruction } else if (condition) { instruction } else { instruction }`

Pretty simple but impactful! Here is a petit example:

```
C/C++
#include <stdio.h>

int main() {
    int num1;
    int num2;
```

```
    char operator;
    int result;

    printf("please insert your first number gently : ");
    scanf("%d", &num1);
    getchar(); // this line swallows the "enter" key you pressed

    printf("nice, now your operator(+ | - | * | /): ");
    operator = getchar();

    printf("now the second number : ");
    scanf("%d", &num2);

    if (operator == '+') {
        result = num1 + num2;
        printf("%d\n", result);
    } else if (operator == '-') {
        result = num1 - num2;
        printf("%d\n", result);
    } else if (operator == '*') {
        result = num1 * num2;
        printf("%d\n", result);
    } else if (operator == '/' && num2 != 0) {
        result = num1 / num2;
        printf("%d\n", result);
    } else {
        printf("booomboclaaaat\n");
    }

    return 0;
}
```

> And finally with that, we'd have written our first ever primitive calculator, quite classic, and brings nostalgia honestly, much love for the craft, man… much love!

—

Now now, your mission after this big chunky ahh chapter is to make your own program, could be a calculator, it could be a metric converter, could be multiplication table, could be an interactive interface, could be perhaps an age authenticator checker, maybe even a star drop percentage calculator (i made that once), literally anything at this point…

—